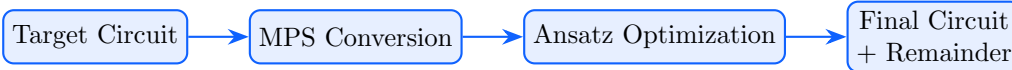


Week 4: Hamiltonian Simulation Workflow

AQC-Tensor + Qiskit Pattern for Time Evolution

quantum.cloud.ibm.com/docs — Hamiltonian Simulation Function Template



What is Hamiltonian Simulation?

- Simulate time evolution $|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle$
- Quantum computers are natural simulators for quantum systems (Feynman's vision)
- Applications: condensed matter physics, chemistry, materials science

Trotterization

- Split Hamiltonian: $H = \sum_k H_k$
- First-order (Lie-Trotter): $e^{-iHt} \approx \left(\prod_k e^{-iH_k \Delta t}\right)^n$
- Second-order (Suzuki-Trotter): $e^{-iH \Delta t} \approx \prod_k e^{-iH_k \Delta t/2} \prod_k e^{-iH_k \Delta t/2}$ (reversed)

Error scaling table:

Formula	Error	Order
Lie-Trotter	$O(t^2/n)$	1st
Suzuki-Trotter	$O(t^3/n^2)$	2nd

AQC-Tensor Circuit Compression

- Problem: deep Trotter circuits $\rightarrow$ many two-qubit gates $\rightarrow$ noise
- AQC-Tensor: Approximate Quantum Compilation with Tensor networks
- Workflow:
 1. Build target circuit (many Trotter steps)
 2. Convert to Matrix Product State (MPS) via tensor network simulation
 3. Generate ansatz from fewer Trotter steps
 4. Optimize ansatz parameters (L-BFGS-B) to maximize fidelity
 5. Append remainder Trotter steps
- Key metric: fidelity $F = |\langle \psi_{\text{ansatz}} | \psi_{\text{target}} \rangle|^2$
- Uses `quimb` tensor network library and `jax` for autodiff

The Qiskit Pattern Applied

Step	Hamiltonian Simulation
1. Map	Define H , initial state, observables; build Trotter/AQC circuit
2. Optimize	Transpile to ISA; remap observables via <code>.apply_layout()</code>
3. Execute	Run via <code>EstimatorV2</code> with error mitigation
4. Post-proc	Extract $\langle O \rangle$; compare with classical simulation

Error Mitigation Options

- ZNE (Zero-Noise Extrapolation): amplify noise via gate folding, extrapolate to zero noise
- Twirling: randomize gate/measurement errors to convert coherent errors to stochastic
- Measurement mitigation: correct readout errors using calibration circuits
- Default options from the template:
 - `noise_factors`: [1, 2, 3]
 - `extrapolator`: ["exponential", "linear", "fall-back"]
 - `num_randomizations`: 300, `shots_per_randomization`: 100

Qiskit Functions & Serverless

- Package workflows as reusable Qiskit Functions
- Deploy to IBM Quantum Platform via `QiskitServerless`
- Run remotely with `template.run(arguments)`
- Supports dry-run mode (inspect circuit depth before hardware execution)
- Custom pip dependencies via `QiskitFunction(dependencies=[...])`

Experiments

XXZ Spin Chain (4 qubits)

Néel state $|0101\rangle$ evolved under Heisenberg XXZ Hamiltonian. Tracked $\langle Z_i \rangle$ magnetization dynamics. Trotter simulation matches exact diagonalization.

Trotter Error Convergence

Second-order Suzuki-Trotter error scales as $O(1/n^2)$, confirmed numerically. 16 steps sufficient for $< 10^{-3}$ error at $t = 1.0$.

Qiskit Tools Used

- `SparsePauliOp`, `PauliEvolutionGate`, `SuzukiTrotter`, `LieTrotter`
- `generate_preset_pass_manager`, `.apply_layout()`
- `BackendEstimatorV2`, `AerSimulator`
- `qiskit-addon-aqc-tensor`, `qiskit-addon-utils` (remote workflow)
- `QiskitServerless`, `QiskitFunction` (deployment)

Key Takeaways

1. **Trotterization is the bridge.** It maps continuous time evolution to discrete gate sequences, with controllable approximation error.
2. **Depth vs accuracy tradeoff.** More Trotter steps improve accuracy but increase circuit depth and noise on real hardware.
3. **AQC-Tensor compresses circuits.** Tensor network optimization reduces two-qubit gate depth while preserving fidelity of the initial Trotter layers.
4. **Error mitigation is essential.** ZNE, twirling, and measurement mitigation combat hardware noise for reliable expectation values.
5. **Qiskit Functions enable reuse.** Packaging the workflow as a serverless function allows systematic studies across different Hamiltonians and parameters.